

SynapNote 实习项目简历亮点与面试讲解

营销落地页 转化归因 运营数据面板 Shopify 维护

这份手册用于把简历中的两条技术亮点还原成可以解释、可以画图、也经得住追问的项目故事。内容以两个本地仓库的现有代码和提交记录为依据，并专门标出不能由代码证明的结论，避免在面试中把实现边界说大。

适用场景

- 面试前快速恢复项目记忆，并形成 30 秒、1 分钟和 3 分钟三种讲法。
- 回答 PC 与 Mobile 架构、UTM 归因、Session、PostHog、CVR 和后台表格等技术追问。
- 识别简历表述中容易被面试官抓住的夸大点，并准备诚实、专业的修正口径。

代码范围

- 营销落地页仓库 D:\synapnote-landingpage
- 运营数据面板仓库 D:\vue-landingPageFeedbackDataTest
- Shopify 维护未在上述仓库中留下代码，因此对应章节只提供知识框架，不代替真实经历。

生成日期 2026 年 9 月 14 日

阅读路线

目标	建议阅读章节	预计时间
马上面试	第一章 第二章的项目介绍 第八章高频问答 第十章红线	30 分钟
系统复习	按章节顺序通读并照着代码路径核对	2 至 3 小时
准备深挖	重点看第五章数据链路 第六章面板 第九章八股	1 至 2 小时

目录

1. 结论与推荐简历表述
2. 项目全貌和一分钟介绍
3. 营销落地页的 PC 与 Mobile 独立渲染
4. 邮箱与反馈转化入口
5. UTM Session PostHog 和 Pixel 数据链路
6. 运营数据分析面板
7. Shopify 日常维护的安全讲法
8. 高频项目追问与参考回答

9. 相关前端和数据八股
10. 面试红线与改进方案
11. 代码证据索引和复习清单

一 结论与推荐简历表述

现有代码能够直接支持的三项贡献

1. 完成 Vue 3 营销落地页的 PC 和 Mobile 两套视图，入口根据 User Agent 只挂载其中一套 DOM；移动端后来从 rem 适配迁移到构建期 px 转 vw。
2. 实现广告参数解析和上报，通过后端返回的 session_id 串联访问与邮箱或反馈提交，并在业务提交成功后发送 PostHog 和 Facebook Pixel 事件。
3. 搭建 Vue 3 加 Element Plus 的运营面板，支持多维筛选、分类统计、CVR、排序、分页、批量删除和 CSV 导出。

最需要校准的地方是三个词：封装模块、实时和 ROI。当前 UTM 逻辑分别写在 PC 与 Mobile 组件内部，并非真正复用的独立模块；面板通过初始化、筛选和手动刷新重新取数，没有 WebSocket 或定时轮询；代码中也没有广告花费或收入字段，不能直接计算 ROI。面试时主动说清这些边界，可信度会更高。

建议采用的简历版本

- 针对营销落地页在 PC 与 Mobile 端布局和交互差异较大的场景，采用基于 User Agent 的视图分流方案，分别实现两套页面组件；结合构建期 px 转 vw 完成移动端适配，并统一接入邮箱订阅与用户反馈入口。
- 实现广告参数解析与转化归因链路：解析 campaign_name 等投放参数并上报后端，将返回的 session_id 持久化到 localStorage，在邮箱或反馈提交时携带该标识；同时接入 PostHog 自定义事件与 Facebook Pixel Lead 事件，用于关联流量来源、站内转化和投放平台回传。
- 搭建运营数据面板，支持设备、时间、广告活动、素材和落地页变体等维度筛选，基于筛选结果计算 CVR 与转化分层，并提供排序、分页、批量删除和 CSV 导出，辅助运营比较不同投放组合的线索转化表现。

如果你能从实习记录中找回真实的投放数据、转化提升或使用规模，再补充量化结果。当前代码无法证明具体提升百分比，因此不要临时编数字。

原表述逐句可信度审计

原表述	代码证据	面试口径
PC Mobile 双端组件独立渲染	App.vue 使用 v-if 和 v-else 在两套视图间选择	成立 但静态 import 仍可能把两套代码打入包中 只能说 DOM 独立挂载
封装 UTM 参数解析模块	getUrlParams 和 recordUtmParams 在 PC 与 Mobile 中各有一份	改成实现解析与上报逻辑 若坚持说封装需说明只把 API 请求单独封装
自动绑定后端 Session ID	UTM 接口返回 session_id 后写入 localStorage 和 formData	成立 但存在请求未完成就提交以及旧 session 长期复用的边界

原表述	代码证据	面试口径
PostHog 自定义埋点	成功 非 201 异常三类事件 并调用 identify	成立 注意邮箱和反馈属于敏感数据 面试应谈脱敏和合规
实时 CVR	computed 响应 allData 和 total 变化 数据靠加载或刷新更新	建议说响应式计算或刷新后即时更新 不要说实时推送
优化 ROI	有广告维度和转化数据 无成本与收入字段	只能说为评估投放提供数据依据 不能说系统直接计算 ROI

二 项目全貌和一分钟介绍

业务问题

这个项目服务于 SynapNote AI 会议助手的早期获客。广告把用户带到落地页后，团队不仅要展示产品价值，还要知道用户来自哪组广告、是否留下邮箱或反馈，以及不同素材和落地页变体的转化情况。两个仓库分别承担前台承接与后台观察。

完整数据流



这里的关键设计不是某一个 UI 组件，而是统一标识 session_id。UTM 参数描述流量来源，session_id 把来源记录与后续表单记录关联起来，PostHog 描述行为事件，Pixel 把 Lead 回传给广告平台。四者职责不同。

30 秒介绍

我在实习期间参与 SynapNote AI 会议助手的增长前端，主要做营销落地页和运营数据面板。落地页采用 PC 与 Mobile 两套视图，承接邮箱和反馈转化；进入页面后解析广告参数并向后端换取 session_id，提交线索时带上这个标识，同时发送 PostHog 和 Facebook Pixel 事件。后台面板再按设备、广告活动、素材和落地页变体筛选数据，计算 CVR 和不同转化分层，帮助运营比较投放组合。

1 分钟介绍

项目的核心问题是广告点击和最终线索之间缺少稳定关联。我的工作分成三部分。第一，考虑到 PC 和手机的版式差异很大，我分别实现两套页面，由入口基于 User Agent 选择视图；移动端以 440 像素设计稿为基准，在构建阶段把 px 转成 vw。第二，页面加载时读取 campaign_name、adset_name、ad_name 等参数，调用 UTM 记录接口获取 session_id，并存到 localStorage。用户之后提交邮箱或反馈时，业务接口会收到同一个 session_id，成功后再发送 PostHog 事件和 Pixel Lead。第三，我用 Vue 3 和 Element Plus 做了运营面板，支持多维筛选、全量结果统计、前端排序分页和 CSV 导出。当前代码的 CVR 定义是有邮箱或反馈的会话数除以筛选后的总会话数。

面试画图顺序

1. 先画广告链接，写出 campaign adset ad content 四类参数。
2. 画落地页入口，分叉为 PC 与 Mobile，但两端共享同一种业务数据结构。
3. 画 UTM 接口并标注返回 session_id，再画 localStorage。
4. 画邮箱和反馈两个转化动作，它们都携带 session_id。
5. 在提交成功节点分叉到 PostHog 和 Pixel。
6. 最后画后台面板，从筛选条件指向 allData，再指向 computed 统计和 CVR。

三 营销落地页的 PC 与 Mobile 独立渲染

入口层如何选择视图

App.vue 在应用初始化时读取 navigator.userAgent，用正则判断是否为移动设备，再通过 v-if 和 v-else 选择 LandingPage 或 LandingPageMobile。v-if 的语义是条件为假时不创建该分支的组件实例和 DOM，因此运行时页面树中只存在一套视图。

```
const detectDevice = () => {
  const userAgent = navigator.userAgent.toLowerCase()
  return
  /mobile|android|iphone|ipad|phone|blackberry|iemobile|wpdesktop/i.test(userAgent)
}
const isMobile = ref(detectDevice())

<LandingPage v-if="!isMobile" />
<LandingPageMobile v-else />
```

PC 视图本身又拆成 Header 与 FeedbackForm；Mobile 视图则把展示和转化逻辑集中在一个文件中。两端都实现邮箱、反馈、UTM 和埋点逻辑，但组织方式不同。

为什么当时适合拆成两套

- PC 页面大量使用相对定位和绝对定位，视觉元素横向展开；Mobile 页面是窄屏双列卡片和内容流，两端 DOM 结构并不只是尺寸不同。
- 独立组件允许分别调整信息密度、图片尺寸、输入控件和反馈方式，减少复杂条件 class。

- 营销页通常页面数量少、更新围绕特定活动，两套视图的重复成本在可控范围内。

代价同样明确：两端业务逻辑重复，修复埋点、校验或 Session 问题时必须同步改两处。更成熟的做法是保留两套展示组件，把 useAttribution、useLeadForm 和 API 层抽成共享 composable。

移动端从 rem 迁移到 vw

提交记录显示项目先完成 Mobile 功能，随后增加 rem 适配，最后迁移到 vw。当前 Vite 配置使用 postcss-px-to-viewport-8-plugin，以 440 为设计稿宽度，在构建期把匹配到的 px 转为 vw，并通过 selectorBlackList 排除 PC 页面和 Toast 样式。

```
pxtoviewport({
  viewportWidth: 440,
  unitPrecision: 5,
  viewportUnit: 'vw',
  propList: ['*'],
  selectorBlackList: [
    'header-section-pc',
    'landing-content-pc',
    'email-section-pc',
    'features-section-pc',
    'feedback-section-pc',
    'Vue-Toastification'
  ],
  minPixelValue: 1
})
```

你可以把选择理由讲成工程权衡：设计稿基准明确，移动端页面相对封闭，构建期转换让开发仍按 px 写样式，产物直接使用 vw；黑名单保护 PC 样式不被误转。不要把它说成性能大幅提升，因为仓库没有性能测试数据。

这个方案的边界

问题	当前行为	更成熟的方案
UA 识别错误	折叠屏 平板 桌面模式可能被误判	用 CSS 容器查询或 matchMedia 决定布局 服务端渲染时结合 Client Hints
窗口缩放	只在初始化时检测 注释掉了 resize 监听	若确需动态切换 监听 matchMedia 并清理监听器
代码体积	两套组件是静态 import v-if 不等于按需下载	defineAsyncComponent 或动态 import 形成独立 chunk
业务重复	UTM 和表单逻辑在两端复制	提取共享 composable 展示组件只保留 DOM 和样式
SEO 与首屏	纯客户端判断 初始 HTML 不含业务内容	需要 SEO 时考虑 SSR 或静态生成

动态 import 的改造示例

当前 App.vue 顶部使用的是静态导入：

```
import LandingPage from './views/landingPage.vue'
import LandingPageMobile from './views/landingPageMobile.vue'
```

即使模板使用 `v-if`，两个模块仍然在入口模块中被静态引用。`v-if` 可以保证运行时只创建一套组件实例和 DOM，但它本身不等于按需加载。可以使用 `defineAsyncComponent` 配合箭头函数中的动态 `import()`：

```
<script setup>
import { defineAsyncComponent, ref } from 'vue'

const LandingPage = defineAsyncComponent(() =>
  import('./views/landingPage.vue')
)

const LandingPageMobile = defineAsyncComponent(() =>
  import('./views/landingPageMobile.vue')
)

const detectDevice = () => {
  const userAgent = navigator.userAgent.toLowerCase()
  return
  /mobile|android|iphone|ipad|phone|blackberry|iemobile|wpdesktop/i.test(userAgent)
}

const isMobile = ref(detectDevice())
</script>

<template>
  <LandingPage v-if="!isMobile" />
  <LandingPageMobile v-else />
</template>
```

这里的 `() => import(...)` 是一个加载器函数。初始化 `App.vue` 时只是创建异步组件定义；当 `v-if` 命中某个分支、Vue 准备渲染该组件时，加载器函数才执行。Vite 底层使用 Rollup，会把两个动态导入目标拆成独立的异步 chunk。移动端命中 `LandingPageMobile` 时，浏览器请求 Mobile 对应的 chunk；PC 组件的 chunk 不会因为模板中存在另一个分支就立即执行。

当前 import 和动态 import 到底有什么区别

可以把过程拆成两个独立问题：

1. **模块什么时候被浏览器加载和执行。这是静态 `import` 与动态 `import()` 的区别。
2. **组件是否创建实例并生成 DOM。这是模板中 `v-if` 与 `v-else` 的区别。

当前项目只处理了第二个问题。`v-if` 保证 PC 和 Mobile 不会同时渲染，但没有明确把两个页面模块拆成按需加载资源。

对比项	当前项目的静态 import	箭头函数中的动态 import
写法	<code>import LandingPage from './views/landingPage.vue'</code>	<code>defineAsyncComponent(() => import('./views/landingPage.vue'))</code>
依赖关系建立时间	构建时即可确定，属于入口的静态依赖	运行到加载器函数时才请求目标模块
Vite 构建行为	通常进入入口依赖图，可能合并到初始 JS 或由构建器优化	通常为目标模块生成单独的异步 chunk
浏览器首次访问	初始资源通常已经包含或引用两端代码	先加载入口，只在命中端需要渲染时请求对应 chunk
返回结果	直接得到组件模块，可以同步注册	<code>import()</code> 返回 Promise，组件模块异步到达
与 v-if 的关系	<code>v-if</code> 只阻止未命中组件实例化，不能撤销顶部 import	<code>v-if</code> 命中时才触发异步组件的 loader
首次渲染体验	不需要等待额外组件请求，但首屏资源可能更大	初始包可变小，但命中组件需要一次异步请求，可配置 loading 状态
适用场景	首屏一定使用、体积小的组件和公共模块	路由页面、弹窗、富文本编辑器、PC Mobile 二选一等非必需模块

当前项目的执行时间线

浏览器访问页面

- > 加载入口 JS
- > 执行 `App.vue` 模块
- > 顶部两个静态 `import` 已属于 `App.vue` 的依赖图
- > `detectDevice` 判断 PC 或 Mobile
- > `v-if` 只创建命中端的组件实例和 DOM

以手机访问为例，`LandingPage` 不会生成 PC 页面 DOM，也不会执行它的 `setup` 和 `onMounted`；但因为它在文件顶部被静态导入，不能仅根据 `v-if` 断言浏览器完全没有获取 PC 相关代码。

改成动态 import 后的执行时间线

浏览器访问页面

- > 加载较小的入口 JS
- > 执行 `App.vue` 模块
- > 创建两个异步组件定义，此时箭头函数中的 `import()` 尚未执行
- > `detectDevice` 判断为 Mobile
- > `v-if` 准备创建 `LandingPageMobile`
- > Vue 调用 `() => import('./views/landingPageMobile.vue')`

- > 浏览器请求 Mobile 异步 chunk
- > 模块到达后创建 Mobile 组件实例和 DOM

如果当前访问的是手机，PC 组件的加载器函数没有被调用，所以对应的 PC 异步 chunk 通常不会在首屏请求。以后条件切换到 PC 时，Vue 才会调用 PC 的加载器；已经成功加载过的异步组件结果会被缓存，通常不会每次切换都重新下载。

为什么要包一层箭头函数

下面两种写法的执行时机不同：

```
// import() 立即执行，马上开始请求模块。
const mobileModulePromise = import('./views/landingPageMobile.vue')

// 这里只创建函数。只有调用 loadMobile() 时，import() 才执行。
const loadMobile = () => import('./views/landingPageMobile.vue')
```

`defineAsyncComponent` 需要接收的是一个加载器函数，所以使用第二种形式：

```
const LandingPageMobile = defineAsyncComponent(
  () => import('./views/landingPageMobile.vue')
)
```

箭头函数不是分包的根本原因，真正触发代码分割的是动态 `import()`；箭头函数的作用是把这次动态导入推迟到 Vue 调用 loader 时再执行。

一个更容易记住的类比

静态 `import` 相当于出门前把 PC 和 Mobile 两套工具都装进车里，到了现场只拿出其中一套使用。`v-if` 决定拿出哪套工具，但两套工具已经随车出发。

动态 `import()` 相当于先到现场判断需要哪套工具，再通知仓库只送这一套。入口携带的东西更少，但需要等待一次配送。`loadingComponent` 就是等待配送时展示的状态。

面试时的简短回答

当前项目在 `App.vue` 中静态导入两套页面，再用 `v-if` 决定实例化哪一套，所以运行时只有一套 DOM，但不能据此认为另一套代码没有进入首屏依赖。改成 `defineAsyncComponent(() => import(...))` 后，`import()` 被包装为加载器，只有对应分支准备渲染时才执行。Vite 通常会据此生成异步 chunk，从而减少未命中端的首屏资源；最终效果要通过构建产物和 Network 面板验证。

如果希望加载失败时有提示、加载较慢时有占位内容，还可以传入配置对象：

```
import { defineAsyncComponent } from 'vue'
import LoadingView from './components/LoadingView.vue'
```



```
import LoadErrorView from './components/LoadErrorView.vue'

const LandingPageMobile = defineAsyncComponent({
  loader: () => import('./views/landingPageMobile.vue'),
  loadingComponent: LoadingView,
  errorComponent: LoadErrorView,
  delay: 200,
  timeout: 10000
})
```

验证时不能只看源码。运行 `npm run build` 后，应检查 `dist/assets` 和 Vite 输出清单：正常情况下会看到入口文件，以及分别对应 PC、Mobile 页面的一组异步 JS/CSS 文件。还可以用浏览器 Network 面板分别模拟 PC 和 Mobile 首次访问，确认首屏只请求命中端的异步 chunk。若要进一步查看每个依赖占用的体积，可接入 `rollup-plugin-visualizer` 生成构建分析图。

需要注意，动态 import 只解决加载时机和初始包体积问题，不会自动消除两端重复的 UTM、表单和埋点代码。如果相同依赖被两个异步组件使用，Rollup 可能把它提取到共享 chunk；是否真正减少首屏传输量，仍应以构建输出和 Network 实测为准。

这一亮点的标准回答

两端设计并不是同一 DOM 的简单缩放。PC 端有横向和绝对定位布局，Mobile 端是窄屏内容流，所以入口根据 User Agent 只挂载一套组件，避免同时维护大量条件样式。移动端以 440 像素设计稿为基准，通过 PostCSS 在构建时将 px 转为 vw，并用选择器黑名单保护 PC 样式。这个方案适合页面数量少、两端差异大的营销页；它的缺点是业务逻辑重复和 UA 识别不绝对可靠，后续我会把归因与表单逻辑提取成共享 composable，并用动态 import 降低初始包体积。

四 邮箱与反馈转化入口

两个入口为什么分开

邮箱代表可持续触达的销售线索，反馈代表需求信号。代码允许用户只提交邮箱，也允许只提交反馈，两种 payload 都附带 session_id。后端可以按照同一会话合并记录，而前端用 submit_type 区分转化类型。

入口	前端校验	提交字段	成功事件
邮箱	非空 简单邮箱正则 v-model.trim	email session_id	feedback_submitted submit_type 为 email Pixel Lead
反馈	trim 后非空	content session_id	feedback_submitted submit_type 为 content Pixel Lead

通用发送函数

sendData 把两类提交共用的网络请求、Toast 和埋点放在一起。接口返回 201 时记录成功；返回其他状态时发 feedback_submit_failed；请求抛出异常时发 feedback_submit_error。这个结构把业务成功与网络成功区分开。

```
const res = await createFeedbackAPI(data)
if (res.status === 201) {
  posthog.capture('feedback_submitted', { session_id, submit_type, ... })
  if (data.email) posthog.identify(data.email, { ... })
  trackLead({ content_category: 'Lead', value: 1.00, currency: 'USD' })
} else {
  posthog.capture('feedback_submit_failed', { error_status: res.status, ... })
}
```

可解释的工程细节

- form 使用 submit.prevent，避免浏览器默认刷新页面；输入使用 v-model.trim，先清理首尾空格。
- email 与 content 分别构造对象，避免向不接受空字符串的后端发送无效字段。
- Axios 实例统一配置 api.synapnote.ai 和 5 秒超时，两个业务接口分别封装在 createFeedback.js 与 recordUtmParams.js。
- PC 使用 vue-toastification，Mobile 使用 Vant Toast，体现两套展示组件可独立选择交互库，但也增加一致性维护成本。

可以继续改进的地方

- 增加 isSubmitting，在请求期间禁用按钮，避免连点造成重复线索和重复埋点。
- 让 handleEmailSubmit 和 handleContentSubmit await sendData，并在成功后清空对应输入框。
- 在请求中加入幂等键，或由后端对 session_id 加转化类型做去重。
- 不要仅靠前端正则验证邮箱，后端仍需做格式、长度、频率限制和垃圾内容过滤。
- 错误事件不要直接上传完整异常或用户输入，应定义白名单字段和脱敏规则。

五 UTM Session PostHog 和 Pixel 数据链路

先把四个概念分清

概念	在本项目中的职责	不负责什么
广告参数	描述用户由哪个活动 广告组 广告和内容进入	本身不能唯一标识一次访问
session_id	后端生成的会话标识 关联来源记录和后续表单	不是登录态 也不是安全认证凭据
PostHog	记录成功 失败 异常等产品行为 identify 关联用户	不能替代业务数据库
Facebook Pixel	在转化成功后回传标准 Lead 事件	当前调用没有携带 session_id 不能直接证明跨系统——join

参数解析

getUrlParams 使用 URLSearchParams 读取查询字符串，并始终加入 device。只有对象键数大于 1 时才认为存在广告参数并请求后端，从而避免普通直接访问也创建 UTM 记录。

URL 字段	语义	面板字段
campaign_name	广告活动	adCampaign
adset_name	广告组	adGroup
ad_name	具体广告	adName
ad_content	素材或文案方向	adContent
landingpage_content	落地页内容变体	landingPageVariant 需由后端映射
landingpage_topic	落地页主题	landingPageTheme 需由后端映射
device	PC 或 mobile	deviceType 需由后端格式化

这些名称不是经典的 utm_source、utm_medium、utm_campaign，而是项目与投放平台约定的自定义字段。面试时可以统称投放归因参数，但被问到标准 UTM 时应说清差异。

时序拆解

1. 组件挂载后先读取 localStorage 中已有的 session_id，恢复到 formData。
2. 解析当前 URL。如果只有 device，没有广告字段，则不调用 UTM 接口。
3. 存在广告字段时，POST 到 /api/v1/utm/track。
4. 后端返回 session_id，前端同时写入 localStorage 与当前表单状态。
5. 用户提交邮箱或反馈，POST 到 /api/v1/feedback/，payload 携带 session_id。
6. 业务接口返回 201 后，PostHog 记录 feedback_submitted，并以 submit_type 区分 email 或 content。
7. 如果 payload 有邮箱，PostHog identify 把匿名行为与邮箱身份关联。
8. 同一成功节点向 Facebook Pixel 发送 Lead 事件。

Session 方案为什么有效

广告参数只在首次进入的 URL 上，后续刷新或再次提交时可能已经没有查询参数。localStorage 让 session_id 跨刷新保存，两个独立表单动作都带同一个标识，因此后端可以从 UTM 访问记录查到来源，再把邮箱和反馈拼到同一条会话链路上。

Session 方案有哪些漏洞

风险	原因	回答中的改进
提交竞态	recordUtmParams 是异步调用 挂载后未阻止用户立即提交	归因初始化增加 attributionReady Promise 提交前等待或禁用按钮

风险	原因	回答中的改进
旧归因污染	无 UTM 的再次访问会继续使用旧 localStorage	给 session 增加 TTL 来源版本和 landing timestamp
新广告覆盖	带新参数访问会覆盖旧 session 归因策略未显式定义	明确采用 last touch 或保存 first touch 与 last touch 两套字段
多标签页覆盖	localStorage 在同域标签页共享	会话级需求用 sessionStorage 长期访客 ID 再单独存 localStorage
接口失败静默	recordUtmParams catch 中没有用户或监控反馈	记录受控错误事件并允许无归因提交 后端标记 unattributed

PostHog 事件设计

事件	触发条件	关键属性	用途
feedback_submitted	业务接口返回 201	session_id submit_type has_email has_feedback feedback_length	计算转化 区分邮箱与反馈
feedback_submit_failed	收到非 201 响应	session_id submit_type error_status	观察业务失败状态
feedback_submit_error	请求抛出异常	session_id submit_type error_message	定位网络或运行时异常

事件命名集中在 feedback，但 email 提交也使用 feedback_submitted。这样实现简单，却会增加分析理解成本。更清晰的方案是统一命名 lead_submitted，再用 lead_type 区分，或者拆成 email_submitted 与 feedback_submitted，并建立事件字典。

隐私和合规是必答项

当前代码把原始 email、feedback_content 和 error_message 发给 PostHog，并直接用邮箱作为 distinct_id。这些字段可能包含个人信息或业务敏感内容。面试时不要为现状辩护，应说明生产方案需要征得用户同意、减少采集字段、对邮箱做不可逆哈希或使用内部 user_id、限制属性白名单、设置保留周期，并确认第三方数据处理协议与地域要求。

为什么还要 Facebook Pixel

PostHog面向站内行为和产品分析；Pixel 面向广告平台优化。代码只在业务接口成功后触发 Lead，比单纯点击按钮更接近有效转化。value 固定为 1 美元只是事件参数，不能等同于真实用户价值。若要提高可靠性，可以增加服务端 Conversions API，并通过 event_id 对浏览器 Pixel 与服务端事件去重。

全链路这个词怎么说才严谨

可以说前端建立了从广告参数到业务提交，再到分析事件和广告回传的链路。不要说已经完成跨平台数据仓库的——核对，因为当前代码只能看到 session_id 被发送给业务后端和 PostHog，Pixel 事件没有 session_id，仓库也没有后端表结构、PostHog 漏斗配置或广告成本回流。

六 运营数据分析面板

面板解决什么问题

落地页产生的会话记录包含来源、设备、落地页版本、邮箱和反馈。面板把这些字段变成可筛选、可统计、可导出的运营视图，让运营人员比较不同投放组合，而不需要直接查询数据库。

页面状态结构

状态	作用	为什么需要
tableData	当前页表格数据	减少表格一次渲染的行数
allData	当前筛选条件下的全量数据	用于顶部统计 排序 前端分页和导出
filters	设备 时间 广告和落地页条件	统一拼装查询参数
pagination	页码 每页数量 总数	驱动 Element Plus 分页器
sortProp 和 sortOrder	当前排序列与方向	对 allData 排序后重新切页
selectedRows	用户勾选的记录	批量删除和导出选中项

CVR 的准确定义

当前代码把发生邮箱提交或反馈提交的会话视为转化。分母是筛选后的总记录数，分子是 email 或 feedback 不等于 未提交 的记录数。公式如下。

```
CVR = 有邮箱或反馈的会话数 / 筛选后的总会话数 * 100%

const cvr = computed(() => {
  return total.value > 0
    ? ((hasEmailOrFeedback.value / total.value) * 100).toFixed(2)
    : '0.0'
})
```

例如筛选某个广告素材后有 200 次会话，其中 30 条有邮箱、10 条只有反馈、5 条同时有邮箱和反馈。只要每条记录按会话唯一，分子是至少出现一种转化的记录数，不应把同时提交两种内容的会话重复计算。假设去重后共有 40 个转化会话，CVR 就是 20%。

四类转化分层

分类	判断条件	业务含义
仅 UTM	邮箱和反馈都未提交	只有访问 没有显式线索
仅邮箱	有邮箱 无反馈	可触达线索
仅反馈	无邮箱 有反馈	有需求信号但无法直接触达
邮箱加反馈	邮箱和反馈都有	同时具备联系方式和需求信息

筛选如何驱动统计更新

任一筛选条件变化时，handleFilterChange 把页码重置为 1 并调用 loadData。loadData 组合 pagination 和 filters 请求 getFeedbackData，更新 allData 与 total，再执行排序和分页。computed 依赖 allData 或 total，因此分类统计与 CVR 会自动重新计算。这里的实时是 Vue 状态变化后的响应式更新，不是服务端推送。

排序和分页

代码选择对 allData 先复制、再排序，最后用 slice 切出当前页。createTime 会先转成 Date，避免按字符串错误排序。取消排序时重新加载数据恢复原始顺序。这个方案对几百或几千条数据简单直接，但它要求前端拿到筛选后的全量记录。

当数据量增长时，应把排序、分页和统计下沉到后端。列表接口只返回当前页，统计接口返回 count 与 CVR，导出则由后端生成文件或异步任务。这样可以降低网络传输、浏览器内存和 $O(n \log n)$ 排序成本。

导出与删除

- 导出优先使用 selectedRows；没有选中时导出当前筛选条件下的 allData。每个字段用双引号包裹，并将内部双引号替换为两个双引号，减少逗号和引号破坏 CSV 结构。
- CSV 前添加 UTF 8 BOM，避免 Windows Excel 打开中文乱码。按钮虽然写导出 Excel，实际格式是 CSV，不是 xlsx。
- 批量删除从 selectedRows 提取 id，调用删除接口后重新加载。删除全部有二次确认，但生产环境仍应增加权限、审计日志和更强确认。

当前仓库的模拟数据边界

src/api/feedback.js 明确写明，实习结束后真实接口不可调用，因此当前仓库用内存和随机数据模拟后端行为。它先尝试 /api/feedback，请求失败后生成 62 条模拟记录，并在 cachedAllData 上完成筛选和删除。这证明你保留或重建了前端交互逻辑，但不能用当前运行结果证明生产数据规模、实时接口或线上效果。

ROI 和其他指标

指标	公式	当前代码能否计算
CVR	转化会话数 / 访问会话数	可以 但取决于每条记录是否代表唯一会话
CPA	广告花费 / 转化数	不可以 缺少广告花费
ROAS	广告收入 / 广告花费	不可以 缺少收入与花费
ROI	收益减成本后再除以成本	不可以 缺少收益和完整成本

因此更准确的说法是面板为投放效果比较和 ROI 分析提供转化侧数据。只有广告平台成本和后续商业价值回流后，才可以计算 CPA、ROAS 或 ROI。

数据面板的一分钟回答

我用 Vue 3 和 Element Plus 做了一个运营反馈面板。筛选条件覆盖设备、时间、广告活动、广告组、具体广告、内容方向和落地页变体。请求完成后，allData 保存当前筛选结果的全量记录，tableData 保存当前页；computed 根据 allData 统计仅访问、仅邮箱、仅反馈、邮箱加反馈四类数据，并计算至少有一种转化的会话占比。排序在全量结果上完成后再分页，导出会处理 CSV 转义和 UTF 8 BOM。这个前端聚合方案适合早期小数据量；生产数据扩大后应由后端返回分页列表和聚合统计。

七 Shopify 日常维护的安全讲法

两个本地仓库没有 Shopify Theme、Liquid、Admin API、Storefront API 或应用配置代码。因此本章不能替你认定具体做过哪些操作。面试时只讲你能回忆并举例的任务，不要把日常维护说成独立搭建跨境电商平台。

推荐表述模板

同时承担 Shopify 站点的日常维护，主要处理我实际做过的商品与页面内容更新、主题样式调整、应用配置、基础 SEO、活动上线或问题排查。每次修改先在主题副本或预览环境验证，再发布到线上，并关注移动端展示、结账链路和第三方应用兼容性。

上面方括号中的任务必须换成你的真实经历。最好准备一个具体例子，说明问题、你改了什么、如何验证、如何回滚。没有例子时，宁可把表述缩小为协助维护。

常见维护知识

主题	面试应知道的要点
Theme 与 Liquid	Liquid 负责模板渲染 section 和 block 让商家配置页面 修改前复制主题并预览
商品与集合	Product Variant Collection Inventory 和 Metafield 是常见数据对象
应用影响	第三方 App 可能注入脚本和 App Embed 影响性能 样式和结账流程
SEO	title description canonical robots sitemap 结构化数据 图片 alt 和重定向
性能	压缩图片 延迟非关键脚本 减少应用数量 检查 LCP CLS INP
发布安全	主题副本 预览链接 核心路径回归 定时发布 保留上一版本以便回滚

Shopify 高频问答

问 Liquid 是什么

答 Liquid 是 Shopify 的模板语言，用对象、标签和过滤器把商品、集合、购物车等数据渲染成 HTML。它主要在服务端生成页面结构，前端 JavaScript 再负责交互。

问 修改线上主题前怎么降低风险

答 先复制当前主题，在副本中修改并通过预览链接验证首页、商品详情、集合、购物车和移动端。上线前记录变更，避开高峰，发布后做核心路径回归；出现问题可快速切回旧主题。

问 Shopify 站点突然变慢怎么排查

答 先对比近期主题和 App 变更，再用浏览器 Performance、Network 和 Lighthouse 找 LCP 资源、长任务、第三方脚本和布局偏移。图片过大就调整尺寸和格式；App 注入过多则关闭 Embed 或移除无用脚本；修改后用相同页面和网络条件复测。

问 如何理解 Product 和 Variant

答 Product 是商品主体，Variant 是尺寸、颜色等可售组合。价格、SKU、条码和库存通常落在 Variant 层。维护时要避免只更新商品标题却漏掉变体价格或库存。

问 Webhook 有什么用

答 Webhook 在订单创建、商品更新等事件发生时由 Shopify 主动通知外部系统，减少轮询。接收端需要校验 HMAC、快速返回、异步处理、幂等去重并设计重试。

八 高频项目追问与参考回答

架构与移动端

问 为什么不用一套响应式 DOM

答 因为 PC 和 Mobile 的内容组织与定位方式差异较大，复用同一 DOM 会引入大量条件样式。我选择两套视图换取更直接的视觉控制，但共享逻辑没有完全抽出，这是当前实现最明显的维护成本。

问 v-if 是否意味着移动端不会下载 PC 代码

答 不能直接这样说。v-if 保证未命中的组件不创建实例和 DOM，但当前两套组件是静态 import，打包器可能把它们放进同一初始依赖图。要做到按端下载，需要动态 import 或 defineAsyncComponent，并通过构建产物验证 chunk。

问 为什么用 User Agent 而不是屏幕宽度

答 当时的目标是按设备进入两套固定布局，并且注释中明确取消了 resize 切换。UA 实现简单，但平板、桌面模式和折叠屏会误判。若重新设计，我会优先让 CSS 决定布局；只有组件行为确实不同才使用 matchMedia 或服务端设备提示。

问 为什么从 rem 改成 vw

答 仓库提交记录显示先做过 rem，后来改为 PostCSS 构建期 px 转 vw。对于 440 像素基准的单页设计，开发按设计稿写 px 更直观，构建后浏览器直接使用 vw。选择器黑名单用于保护 PC 样式。

问 scoped CSS 的原理

答 Vue SFC 编译器会给当前组件 DOM 和 CSS 选择器添加同一个 data 属性，使规则只匹配当前组件。它不是 Shadow DOM；对子组件根节点、深度选择器和动态插入内容仍需理解其边界。

归因与埋点

问 什么是 UTM 参数

答 UTM 参数是添加在落地页 URL 查询字符串中的一组来源标签，用来描述一次访问是由哪个渠道、哪次广告活动或哪条素材带来的。用户点击带 UTM 参数的广告链接后，网站可以读取这些字段，并把后续浏览、注册或提交行为与流量来源关联起来。

例如一条标准链接可能是：

```
https://example.com/landing
?utm_source=facebook
&utm_medium=paid_social
&utm_campaign=meeting_assistant_launch
&utm_content=painpoint_video_a
```


其中 `utm_source` 表示来源平台，`utm_medium` 表示流量媒介，`utm_campaign` 表示广告活动，`utm_content` 常用于区分素材或文案版本。UTM 参数本质上只是普通 URL 参数，不会自动完成统计；页面、分析 SDK 或后端必须读取并保存它们，之后才能做归因分析。

面试时可以直接回答：

UTM 参数就是放在广告落地页 URL 上的来源标签。它告诉系统用户从哪个渠道、广告活动和素材进入。前端解析这些参数并保存，等用户提交邮箱或完成其他转化时，就能把转化与对应流量来源关联起来，从而比较不同投放组合的效果。

问 标准 UTM 通常有哪五个参数

答 常见的五个参数如下。前三个是最核心的来源维度，后两个用于进一步区分关键词和素材。

参数	含义	示例
<code>utm_source</code>	流量来源或平台	<code>facebook google newsletter</code>
<code>utm_medium</code>	流量媒介或投放类型	<code>cpc paid_social email</code>
<code>utm_campaign</code>	广告活动名称	<code>meeting_assistant_launch</code>
<code>utm_term</code>	付费搜索关键词等定向信息	<code>ai_meeting_notes</code>
<code>utm_content</code>	区分素材、文案或按钮版本	<code>painpoint_video_a</code>

项目代码读取的不是完整标准命名，而是 `campaign_name`、`adset_name`、`ad_name`、`ad_content`、`landingpage_content` 和 `landingpage_topic` 等业务自定义参数。它们与标准 UTM 的目标相同，都是描述投放来源，但字段名来自项目与广告平台或后端的约定。面试时不要说这些就是标准五参数，可以说它们是项目中的 UTM 类归因参数或自定义投放参数。

问 你在项目里具体怎么处理 UTM 参数

答 页面挂载后，我使用 `window.location.search` 取得查询字符串，再交给 `URLSearchParams` 解析。代码按白名单读取广告活动、广告组、广告名称、素材内容、落地页内容和主题，同时补充设备类型。只有检测到真实广告字段时才调用 UTM 记录接口，后端保存来源信息并返回 `session_id`。前端把它写入 `localStorage` 和表单状态，之后提交邮箱或反馈时携带同一个 `session_id`，后端便能把来源记录与转化记录关联。

```
const params = new URLSearchParams(window.location.search)
const utmParams = { device: getDeviceType() }

const campaignName = params.get('campaign_name')
const adsetName = params.get('adset_name')
const adName = params.get('ad_name')
const adContent = params.get('ad_content')

if (campaignName) utmParams.campaign_name = campaignName
if (adsetName) utmParams.adset_name = adsetName
if (adName) utmParams.ad_name = adName
if (adContent) utmParams.ad_content = adContent
```

问 UTM 参数和 session_id 有什么区别

答 UTM 参数描述流量从哪里来，`session_id` 标识这一次访问或归因会话。多个用户可能来自完全相同的广告参数，所以 UTM 不能唯一表示一个用户或一次访问；后端为每次会话生成 `session_id` 后，邮箱提交、反馈提交等多个动作都可以通过这个键关联到同一条来源记录。

可以简单记成：UTM 回答“从哪里来”，`session_id` 回答“是哪一次访问”。

问 为什么不一直把 UTM 参数保留在 URL 中

答 URL 参数可能在刷新、跳转、分享链接或用户再次进入时丢失，也不适合让后续每个业务请求重复携带一组来源字段。项目先把 UTM 上报给后端，再用较短的 `session_id` 关联后续动作。这样既减少 payload 重复，也让后端统一维护来源记录。生产方案还应明确 Session 有效期，以及首触点还是末触点归因。

问 如果用户没有 UTM 参数怎么办

答 当前代码只包含 `device` 时不会请求 UTM 记录接口，但会尝试恢复 `localStorage` 中已有的 `session_id`。这能延续之前的归因，也可能把新的一次自然访问错误归到旧来源。更完善的方案会设置有效期，并把无来源访问标记为 `direct` 或 `unattributed`，而不是无限期复用旧 Session。

问 为什么需要 session_id 直接把 UTM 放进提交接口不行吗

答 直接携带 UTM 也能完成简单归因，但每次提交都要重复字段，也难表示一次访问中的多个动作。后端生成 `session_id` 后，来源记录与多个转化动作可以用统一键关联，前端 payload 更稳定。

问 localStorage sessionStorage Cookie 怎么选

答 `localStorage` 跨标签和浏览器重启保留，适合持久访客状态但容易留下旧归因；`sessionStorage` 按标签页会话隔离，适合一次访问；Cookie 可随请求发送并支持 `HttpOnly`、`SameSite`、`Secure` 等属性。这个 `session_id` 不是认证凭证，当前用 `localStorage` 是为了刷新后恢复，但应增加 TTL 和归因策略。

问 首触点和末触点归因有什么区别

答 首触点把转化归给第一次获客来源，适合评估认知渠道；末触点归给转化前最后一次来源，适合评估促成动作。当前代码遇到新 UTM 会覆盖 `localStorage`，更接近末触点，但没有明确 TTL 和历史字段，所以说实现倾向，不能说完整归因模型。

问 为什么只在接口成功后发转化事件

答 如果点击按钮就上报，会把校验失败、网络失败和后端拒绝都算成转化。以业务接口返回 201 为触发点，更接近真实落库的线索。进一步还应由服务端发送权威转化事件，并做去重。

问 capture 和 identify 的区别

答 `capture` 记录一次事件及其属性；`identify` 把匿名 `distinct_id` 与稳定用户标识关联，并设置用户属性。`identify` 不应随便使用邮箱原文，生产环境更适合内部 `user_id` 或哈希标识。

问 为什么既有 PostHog 又有 Pixel

答 PostHog 用于站内漏斗、错误和用户行为分析；Pixel 把 Lead 信号发回 Meta 广告平台，用于广告归因和投放优化。两者目标不同，业务数据库仍是线索事实来源。

问 如何防止重复埋点

答 前端先禁用重复提交；事件带 event_id 或业务请求幂等键；服务端按 session_id、动作类型和时间窗口去重；若同时使用 Pixel 与 Conversions API，则用同一个 event_id 做浏览器端和服务端去重。

问 当前实现有什么隐私风险

答 邮箱、反馈原文和错误消息被发送给第三方分析平台。需要最小化采集、用户同意、字段脱敏、访问控制和数据保留策略。邮箱不应默认充当公开 distinct_id。

数据面板

问 computed 和普通函数有什么区别

答 computed 会追踪响应式依赖并缓存结果，依赖未变化时重复读取不会重新计算。普通函数每次渲染调用都会执行。这里分类数量依赖 allData，CVR 还依赖 total，因此数据更新后自动重算。

问 为什么 loadData 后还要 applySortAndPagination

答 接口返回当前筛选结果的 allData，页面把排序状态应用到这份全量数组，再用 page 和 pageSize 切出 tableData。这样跨页排序一致，但代价是前端必须拿到全量数据。

问 前端分页和后端分页怎么选

答 小数据量、快速原型可以前端全量处理，逻辑迭代快。数据量上升后应采用后端筛选、排序和分页，另外返回聚合统计；否则网络、内存、首屏和导出都会成为瓶颈。

问 为什么导出 CSV 要加 BOM 和转义

答 UTF 8 BOM 能减少 Windows Excel 打开中文乱码。字段可能包含逗号、换行或双引号，因此要用双引号包裹，并把内部双引号写成两个双引号。当前页面已处理双引号，并用 BOM 指定 UTF 8。

问 面板真的实时吗

答 不是推送式实时。computed 对已加载数据是即时响应的，但数据源只在初始化、筛选、翻页或手动刷新时更新。准确说法是刷新后即时统计，若要实时需要轮询、SSE 或 WebSocket。

问 CVR 为什么可能算错

答 前提不成立时会出错，例如一条记录不等于唯一会话、同一 session 有多行、机器人访问未过滤、接口只返回部分 allData，或未提交使用 null 而不是固定字符串。生产环境应按唯一 session 去重，并让后端定义统一的转化字段。

问 如何支持 A B 测试

答 landingPageVariant 和 adContent 可以作为实验维度，但严格 A B 测试还需要随机分流、互斥实验组、样本量、显著性检验和固定观察窗口。当前面板更准确地说是按变体比较，不是完整实验平台。

问 如何把面板改成生产级

答 列表接口后端分页和排序，统计由聚合接口返回；筛选输入做防抖和取消旧请求；权限按角色控制；删除写审计日志；导出改为异步任务；增加空态、错误重试、时区处理和可观测性。

网络与工程化

问 Axios 5 秒超时意味着什么

答 请求超过 5 秒会被客户端中断并进入 catch，但不代表服务端一定停止处理。对提交接口要防止用户重试造成重复写入，因此仍需要幂等设计。

问 为什么使用 Axios 实例

答 集中维护 baseUrl、timeout、请求头和拦截器，业务 API 只表达路径和数据。当前拦截器只是透传，后续可加入请求标识、统一错误映射或认证，但不要在拦截器里吞掉错误。

问 201 与 200 有什么区别

答 200 表示请求成功，201 更明确表示服务器创建了新资源。落地页把 201 当作业务提交成功。更稳妥的前端应以接口契约为准，也可以接受后端定义的成功码，而不是随意假设。

问 CORS 在这个项目里为什么可能出现

答 页面域名和 api.synapnote.ai 很可能不同源，浏览器会执行跨域检查。后端需要允许正确的 Origin、方法和请求头。若携带 Cookie，还要显式允许凭证且不能用通配 Origin。

问 Docker 和 Nginx 做了什么

答 Dockerfile 用 Node 18 Alpine 构建 Vite 产物，再复制到 Nginx 镜像提供静态文件。Nginx 开启 gzip、SPA fallback 和静态资源长期缓存，并配置 User Agent 黑名单。长期缓存适合带 hash 的构建资源，不应盲目给不带版本的文件 immutable。

九 相关前端和数据八股

Vue 3

问 ref 为什么访问时需要 value 模板里为什么不用

答 ref 用对象包装值以便依赖追踪，JavaScript 中通过 value 读写；模板编译会自动解包顶层 ref，因此模板中直接写变量名。

问 computed 为什么不适合有副作用

答 computed 应是基于依赖的纯派生值，可能被缓存并在需要时惰性求值。网络请求、写 localStorage 或修改其他状态应放在事件处理、watch 或生命周期中。

问 watch 和 watchEffect 的区别

答 watch 显式指定依赖，可拿到新旧值并控制 immediate、deep；watchEffect 自动收集同步执行期间访问的依赖，适合依赖关系简单的副作用。筛选请求可用 watch 统一监听 filters，但要防抖和取消旧请求。

问 v-if 和 v-show 的区别

答 v-if 会创建和销毁组件，切换成本较高但初始不渲染未命中分支；v-show 始终渲染，只切换 display。PC 与 Mobile 基本不会频繁切换，所以 v-if 更合适。

问 组件卸载时为什么要清理监听器

答 window 事件、定时器和自建订阅不受组件自动管理，未清理会造成内存泄漏或重复回调。当前 resize 监听已被注释；如果使用 matchMedia，需要在 onBeforeUnmount 移除。

浏览器和 JavaScript

问 URLSearchParams 的优点

答 它按 URL 查询字符串规范解析编码，提供 get、has、append 等接口，比手写 split 更可靠。get 只返回第一个同名值，多值参数应使用 getAll。

问 localStorage 有哪些特性

答 同源、同步、字符串存储、跨标签共享、无自动过期。同步读写过大数据会阻塞主线程；用户可清除，隐私模式和浏览器策略也可能限制它，因此不能作为唯一可靠数据库。

问 浅拷贝为什么用展开运算符

答 sort 会原地修改数组。代码先用 [...allData] 创建新的数组，再排序，避免直接改变响应式源数组的顺序。对象元素仍是同一引用，所以这是浅拷贝。

问 数组 filter 和 sort 的复杂度

答 一次 filter 是 $O(n)$ ，sort 通常是 $O(n \log n)$ ，slice 当前页约 $O(k)$ 。多个 computed 各自 filter 会重复遍历 allData；大数据量可在一次 reduce 中完成所有分类统计，或下沉后端。

问 如何处理请求竞态

答 筛选快速变化时，后发请求可能先返回，随后旧请求覆盖新结果。可以用 AbortController 取消旧请求，或给每次请求递增序号，只接收最后一次响应。

数据分析

问 CVR CPA ROAS ROI 的区别

答 CVR 衡量访问到转化的比例；CPA 是每个转化的广告成本；ROAS 是广告收入除以广告花费；ROI 关注全部收益与成本后的回报。当前面板只能直接算 CVR。

问 什么是 ROI

答 ROI 是 Return on Investment，也就是投资回报率，用来衡量投入一笔成本后获得了多少净收益。常见公式是：

$$\text{ROI} = (\text{投资收益} - \text{投资成本}) / \text{投资成本} \times 100\%$$

例如一次投放总成本为 10,000 元，由这次投放带来的可归因收益为 15,000 元，那么：

$$\text{ROI} = (15,000 - 10,000) / 10,000 \times 100\% = 50\%$$

这表示扣除投入后，净收益相当于投入成本的 50%。如果 ROI 为负数，说明按当前统计口径，收益尚未覆盖成本。

这里必须先约定“收益”和“成本”的口径。成本可能不只包含广告费，还可能包含素材制作、人力、工具或履约成本；收益可以使用收入，也可以更严谨地使用毛利。不同团队口径不同，不能只看到销售额就直接称为 ROI。

问 ROI 和 ROAS 有什么区别

答 ROAS 只关注广告收入与广告花费，公式通常是 $\text{广告收入} / \text{广告花费}$ ；ROI 关注净收益与整体投资成本，公式是 $(\text{收益} - \text{成本}) / \text{成本}$ 。例如广告花费 10,000 元，带来 15,000 元收入，ROAS 是 1.5，也可以写成 150%；如果暂时忽略其他成本，ROI 是 50%。因此两者数字不同，不能混用。

问 你的项目是怎么帮助优化 ROI 的

答 我的前端工作没有直接计算完整 ROI，而是补齐了 ROI 评估中的转化侧数据。UTM 类参数记录广告活动、广告组和素材来源，`session_id` 把这些来源与邮箱或反馈转化关联，运营面板再按投放维度计算 CVR。团队可以据此发现哪些来源带来更多有效线索，把预算从低转化组合调整到高转化组合，从而为降低获客成本和改善 ROI 提供依据。

如果广告平台成本以及线索后续产生的订单、收入或毛利也回流到同一数据体系，才可以进一步按活动计算 CPA、ROAS 和 ROI。当前仓库没有花费与收益字段，所以面试时应使用“辅助评估和优化 ROI”，不要说“我在面板中直接算出了 ROI”。

面试时可以直接回答：

ROI 是净收益相对于投入成本的比例。在这个项目里，我负责的部分主要补齐转化归因：通过投放参数和 `session_id` 知道每条邮箱或反馈来自哪个广告组合，再在面板中比较 CVR。这些数据可以帮助运营调整预算，间接改善 ROI；但完整 ROI 还需要广告成本以及线索后续收入或毛利数据，当前前端面板本身没有直接计算它。

问 如果没有最终收入 怎么评价广告效果

答 可以先使用中间指标，但要明确它们不是最终 ROI。例如用 CTR 看广告吸引力，用落地页 CVR 看访问到线索的效率，用 CPA 或 CPL 看获得一次转化或一条线索的成本，再观察线索到付费客户的转化率。早期产品没有稳定收入时，邮箱和反馈可以作为代理转化指标，但最终仍要与订单、订阅或毛利数据打通。

问 CVR 提升是否一定意味着 ROI 提升

答 不一定。CVR 提升可能来自低质量线索，后续付费率反而更低；也可能为了提高转化而增加了广告成本、优惠或运营成本。ROI 还受到客单价、毛利、履约成本和客户生命周期价值影响。所以 CVR 是重要的过程指标，但不能单独代表商业回报。

问 漏斗分析要注意什么

答 每一层必须有明确事件、去重主体和时间窗口，后层通常应是前层子集。还要处理跨设备、重复事件、机器人流量和延迟上报，否则漏斗会失真。

问 什么是事件属性和用户属性

答 事件属性描述某次动作，例如 `submit_type` 和 `session_id`；用户属性描述较稳定的用户特征，例如注册时间或套餐。把临时行为写成用户属性会造成分析混乱。

问 什么叫幂等

答 同一个业务请求执行一次或多次，最终结果一致。提交线索时可由客户端生成 `idempotency_key`，服务端建立唯一约束；重试不会新增多条记录。

安全与质量

问 前端校验为什么不够

答 前端校验改善体验，但可以被绕过。后端仍需验证类型、长度和格式，限制频率，清理危险内容，并以服务端数据为准。

问 把 API key 写前端安全吗

答 任何进入浏览器包的值都可被用户看到。PostHog 项目 key 通常用于公开采集，不等同服务端密钥，但仍应通过域名、采集规则和平台权限限制滥用；真正的 secret 绝不能放前端。

问 用户反馈如何避免 XSS

答 Vue 插值默认转义文本，但若后续用 v-html 或后台富文本渲染，就必须经过可信的 HTML 清洗。数据库也应保存原始值并在输出端按上下文编码。

问 删除全部为什么危险

答 前端确认框只能防误点，不能提供安全边界。生产环境应限制角色、要求再次输入确认、记录审计、支持软删除或恢复，并由后端验证权限。

十 面试红线与改进方案

不要这样说

高风险说法	问题	更稳妥的说法
v-if 让移动端完全不加载 PC 代码	当前是静态 import 未检查分包	只挂载一套组件和 DOM 动态分包可继续优化
封装了可复用 UTM 模块	解析逻辑在两端重复	实现 UTM 解析与上报 API 请求做了模块化
实时数据大屏	没有推送或轮询	筛选或刷新后响应式更新统计
直接优化了 ROI	没有成本 收入和结果数据	为投放对比和 ROI 分析提供转化侧数据
完整 A B 测试系统	只有变体筛选 没有随机分流和显著性	支持按广告内容和落地页变体比较
PostHog 已实现用户画像	只有少量事件和 identify	建立了转化事件和身份关联的基础
独立负责 Shopify 平台开发	仓库没有证据 简历只写日常维护	按真实任务说协助或负责日常维护

如果重新做一次

- 1. 抽取 useAttribution：统一解析参数、恢复 Session、调用接口、处理 TTL 和归因策略。
- 2. 抽取 useLeadForm：统一校验、提交状态、错误映射、幂等键和成功清理。
- 3. 两套展示组件使用动态 import，并通过构建分析确认分包和资源体积。

例如使用 `defineAsyncComponent(() => import('./views/landingPageMobile.vue'))`。箭头函数让 `import()` 在组件命中渲染分支时才执行，Vite 会为动态导入生成异步 chunk；构建后再结合 `dist/assets`、构建清单和浏览器 Network 面板确认 PC 与 Mobile 资源是否按预期分离。

- 4. 设计事件字典，统一命名、属性类型和版本；删除原始反馈与邮箱，改为内部标识或脱敏字段。
- 5. 增加服务端转化上报和 `event_id` 去重，让业务数据库成为转化事实来源。
- 6. 面板改为后端筛选、排序、分页与聚合；前端增加请求取消、防抖、权限和审计。
- 7. 把广告成本和后续线索价值接入，才进一步展示 CPA、ROAS 和 ROI。

可作为难点故事的三个题目

移动端适配迁移

情境：PC 与 Mobile 设计差异明显，移动端需要覆盖不同宽度。任务：在不影响 PC 样式的前提下，让 440 像素设计稿按视口缩放。行动：先完成独立 Mobile 视图，尝试 rem 后迁移到 PostCSS px 转 vw，并用 `selectorBlackList` 排除 PC 与第三方样式。结果：形成构建期适配方案。这里不要补充没有数据支持的性能提升百分比。

广告来源与线索关联

情境：后端能收到邮箱或反馈，但运营需要知道它来自哪次广告访问。任务：建立稳定关联。行动：解析广告参数，上报后端换取 `session_id`，持久化后随两个转化入口提交；成功后记录 PostHog 和 Pixel。结果：代码层面形成来源到转化的可关联链路。改进点是处理竞态、TTL、隐私与服务端去重。

运营面板的数据建模

情境：运营需要快速比较广告活动、素材和页面变体。任务：提供筛选、统计与导出。行动：用 `filters` 统一查询条件，用 `allData` 支撑 `computed` 分类和 CVR，用 `tableData` 支撑分页展示，并加入排序、删除和 CSV。结果：形成小数据量下可用的分析工具。改进点是数据增长后把全量计算下沉后端。

十一 代码证据索引和复习清单

关键文件

主题	文件与行号	证据
端侧分流	D:\synapnote-landingpage\src\App.vue 3 至 21 行	v-if v-else UA 检测 resize 注释
PC 业务链路	src\components\FeedbackForm.vue 119 至 349 行	参数解析 Session 提交 PostHog Pixel
Mobile 业务链路	src\views\landingPageMobile.vue 23 至 190 行	与 PC 对应的移动端实现

主题	文件与行号	证据
移动端构建适配	vite.config.js 20 至 39 行	440 基准 px 转 vw PC 黑名单
业务 API	src\apis\createFeedback.js src\apis\recordUtmParams.js	反馈与 UTM 两个 POST 接口
HTTP 配置	src\utils\http.js 1 至 18 行	baseUrl 5 秒超时 拦截器
PostHog	src\composables\usePosthog.js 1 至 10 行	初始化配置
Pixel	src\composables\useFbPixel.js	标准事件和 Lead 快捷方法
面板	D:\vue-landingPageFeedbackDataTest\src\components\FeedbackDataManagement.vue	computed 筛选 排序 分页 删除 导出
模拟接口	D:\vue-landingPageFeedbackDataTest\src\api\feedback.js	真实接口不可用后的 mock 与字段模型

提交记录能够支持的工作轨迹

提交	说明	可用于回答
3c4af63	PC 端功能完成	PC 页面实现
294fb77	移动端功能完成	Mobile 页面实现
ef3c1e6	移动端 rem 布局适配	早期适配方案
f0e1488	使用 vw 适配并更新文档	方案迭代和取舍
f8c8c9f	修复移动端标题换行	移动端细节问题
e6283c1	修复时间排序	面板排序 bug 修复

面试前必须补齐的真实信息

- 落地页服务的具体广告平台和国家或地区。代码能证明 Facebook Pixel，不能自动证明所有渠道。
- 线上真实访问量、提交量、CVR 或改版前后对比。没有记录就不写数字。
- 你与设计、后端、运营分别如何协作，接口字段由谁确定，出现过什么联调问题。

- PostHog 中实际配置过哪些漏斗、看板或告警；代码只证明事件发送。
- 数据面板在实习时连接的真实接口形态，以及当前模拟仓库与当时版本的差异。
- Shopify 实际维护过的店铺模块、故障案例、发布流程和你的权限范围。

七天复习清单

天数	任务	验收标准
第 1 天	通读 App.vue FeedbackForm.vue Mobile 视图	能不看稿画出页面和组件关系
第 2 天	手写 UTM 到 session_id 的时序	能解释 localStorage 竞态和归因覆盖
第 3 天	复习 PostHog Pixel 隐私与幂等	能回答 capture identify 去重和 PII
第 4 天	通读数据面板 computed 排序分页和 CSV	能手算一组 CVR 并说出规模瓶颈
第 5 天	补齐 Shopify 真实案例	准备一个问题 修改 验证 回滚故事
第 6 天	按第八章进行追问演练	每题控制在 40 至 90 秒
第 7 天	录制 1 分钟和 3 分钟项目介绍	去掉夸大词 保留技术决策与边界

最后的回答原则

先讲业务问题，再讲你的选择和代码落点，最后主动说一个边界与改进。面试官通常不要求实习项目完美，但会判断你是否理解自己写过的代码。对当前仓库没有证据的生产规模、性能收益和业务结果，应明确说需要以当时的数据或团队记录为准。